



## 1.18 Exploring HttpServletRequest and HttpServletResponse

### HttpServletRequest

- ◆ When you send the request using http protocol that request is called as HttpRequest.
- ◆ HttpRequest contains two parts:
  - HttpRequest Headers.
  - HttpRequest Body.
- ◆ When you send the request with Http method GET then parameter data will be converted to Query String and that Query String will be attached to the URL as follows:

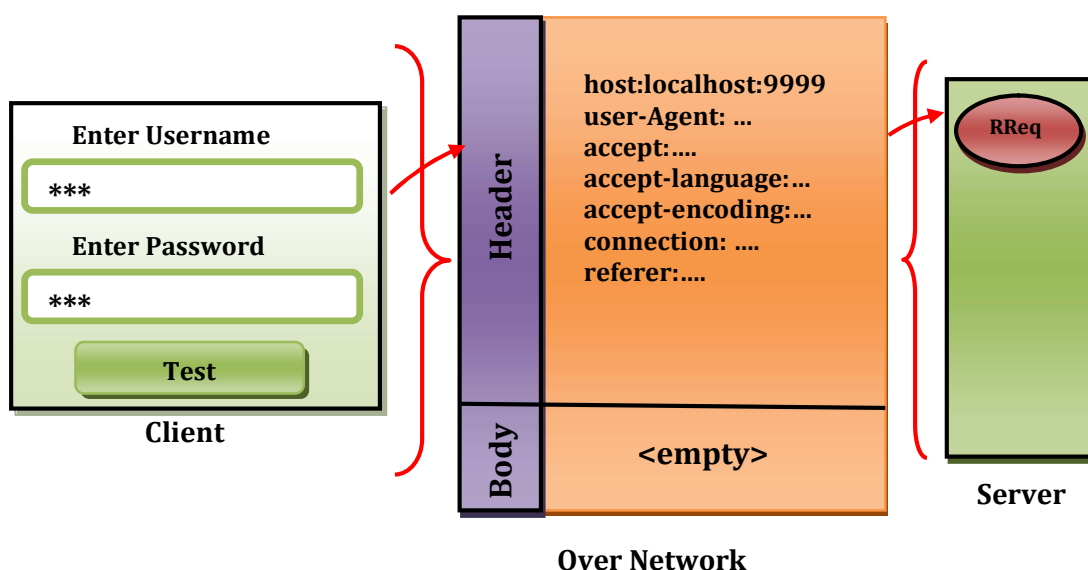
`http://localhost:9999/myjlcapp/login.html`

`http://localhost:9999/myjlcapp/login.jlc?username=sri&password=jlcindia`

|                                                    |              |
|----------------------------------------------------|--------------|
| <code>http://localhost:9999/Lab14/login.jlc</code> | URL          |
| <code>/Lab14</code>                                | CONTEXT PATH |
| <code>http</code>                                  | PROTOCOL     |
| <code>localhost</code>                             | HOST         |
| <code>9999</code>                                  | PORT         |
| <code>/Lab14/login.jlc</code>                      | URI          |
| <code>username=sri&amp;password=jlcindia</code>    | Query String |

- ◆ In the case Http method GET, this data will be placed in HttpRequest Headers and HttpRequest Body is empty.

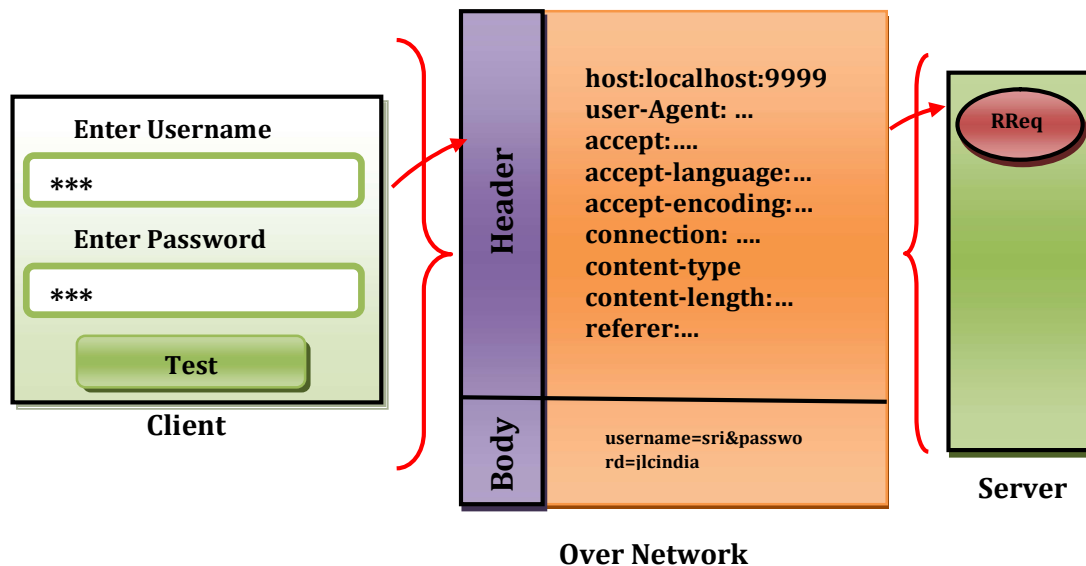
`http://localhost:9999/myjlcapp/login.jlc?username=sri&password=jlcindia`





- ♦ When you send the request with Http method POST then parameter data will be converted to Query String and that Query String will be placed in HttpRequest Body. Then URL is like

`http://localhost:9999/myjlcapp/login.jlc`



| GET                                                                                                | POST                                                                                            |
|----------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| When you send the request with Http method GET, this data will be attached to URL as Query String. | when you send the request with Http method POST, this data will be placed in Http Request Body. |
| Using GET, you can send only limited amount of Data.                                               | Using POST, you can send unlimited amount of Data.                                              |
| GET is not secure because data will be visible in the URL.                                         | POST is more secure.                                                                            |

- ♦ At Serverside, Container is responsible for creating HttpServletRequest and HttpServletResponse objects and Initializes those objects with the data.
- ♦ HttpServletRequest contains the following information
  - Request Parameters
  - Request Headers
  - Request Cookies
  - Other Information.



- ♦ Following are Http Methods supported
  - GET
  - POST
  - PUT
  - DELETE
  - HEAD
  - PATCH
  - TRACE
  - OPTIONS

### **HttpServletResponse**

- ♦ When you receive the response using http protocol that response is called as HttpServletResponse.
- ♦ HttpServletResponse contains two parts:
  - HttpServletResponse Headers.
  - HttpServletResponse Body.

#### **HttpServletResponse Headers.**

Following are some the Response Headers from Server.

200 OK

Access-Control-Allow-Origin: \*

Connection: Keep-Alive

Content-Encoding: gzip

Content-Type: text/html; charset=utf-8

Content-Length:55KB

Last-Modified: Mon, 18 Jul 2016 02:36:04 GMT

Server: Apache

Set-Cookie: mykey=myvalue; Max-Age=31449600;

etc

#### **HttpServletResponse Body.**

- ♦ Any type of output sent by the Server side component will be written to the HttpServletResponse.
- ♦ All MIME types Allowed in HttpServletResponse Body.
- ♦ Some of the MIME Types includes
  - text/html
  - text/plain
  - application/xml
  - application/json
  - application/pdf
  - etc
- ♦ HttpServletResponse contains the following information
  - Response Headers
  - Response Stream in terms of Byte and Character.



## demo1-rest-boot: Spring REST with Boot

### Working Steps:

#### A) Setup the Project

##### 1) Create the maven Project.

- a) Open the Eclipse required Workspace
- b) Select New -> Maven Project.
- c) Check the checkbox for create Simple Project(skip the Archetype selection) and Click on Next Button.
- d) Provide the following  
Group Id: com.coursecube.springboot  
Artifact Id : **demo1-rest-boot**  
Version : 1.0  
packaging : jar  
Name : **demo1-rest-boot**  
and Click On finish button.

##### 2) Remove the following two Source Folders.

- a. src/test/java
- b. src/test/resources

##### 3) Create application.properties under src/main/resources

##### 4) Update **application.properties** as follows

spring.application.name=my-hello-application  
server.port=12345

##### 5) Update **pom.xml** by adding the following dependencies.

```
<dependency>  
<groupId>org.springframework.boot</groupId>  
<artifactId>spring-boot-starter-web</artifactId>  
</dependency>
```

```
<dependency>  
<groupId>org.springframework.boot</groupId>  
<artifactId>spring-boot-devtools</artifactId>  
</dependency>
```



6) You can see the Error in Maven Project.

Fix the Error by updating the Maven Project.

**Select the Project , right click and then select Maven -> Update Project.**

7) Create the Package called **com.jlcindia.springboot** under src/main/java

8) Write Boot Main Class – MyBootApplication.java

**@SpringBootApplication**

```
public class MyBootApplication {  
    public static void main(String[] args) {  
        SpringApplication.run(MyBootApplication.class, args);  
    }  
}
```

9) Run the Boot Application

You will see – Whitelabel Error Page means DS is Ready.

### **B)Develop RESTful Web Service**

10) Write Resource Representation Class called **Hello**

a. **Hello.java** (com.jlcindia.springboot)

11) Create a Resource Controller called HelloController

a. **HelloController.java** (com.jlcindia.springboot)

### **C) Test Rest API with Browser.**

12) Hit the following urls from Browser

http://localhost:12345/myhello

http://localhost:12345/myhello/Srinivas

http://localhost:12345/myhello/name/Srinivas

http://localhost:12345/myhello/name/Srinivas/message/Hello Guys!!!

http://localhost:12345/myhello1/name/Srinivas

http://localhost:12345/myhello1



#### **D) Test Rest API with CURL.**

13) Download CURL from <https://curl.haxx.se/download.html>

You will get zip: curl-7.67.0.zip

14) Extract to E: drive.

Now curl home is E:\curl-7.67.0

15) Goto E:\curl-7.67.0 and type the following

curl -v GET localhost:12345/myhello

curl -v GET localhost:12345/myhello/Srinivas

curl -v GET localhost:12345/myhello/name/Srinivas

curl -v GET localhost:12345/myhello/name/Srinivas/message/Hello Guys!!!

curl -v GET localhost:12345/myhello1/name/Srinivas

curl -v GET localhost:12345/myhello1

#### **E) Test Rest API with Postman**

try yourself

### **Spring Boot-REST Example**

#### **demo1-rest--boot: Files required**

|                           |                           |
|---------------------------|---------------------------|
| 1. MyBootApplication.java | 2. Hello.java             |
| 3. HelloController.java   | 4. application.properties |
| 5. pom.xml                |                           |

#### **1. MyBootApplication.java**

```
package com.jlcindia.springboot;
```

```
import org.springframework.boot.SpringApplication;
```

```
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @company : Java Learning Center
```

```
**/
```

```
@SpringBootApplication
```

```
public class MyBootApplication {
```

```
    public static void main(String as[]) {
```

```
        SpringApplication.run(MyBootApplication.class, as);
```

```
    }
```

```
}
```



## 2. Hello.java

```
package com.jlcindia.springboot;

import javax.xml.bind.annotation.XmlRootElement;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@XmlRootElement
public class Hello {

    private int id;
    private String name;
    private String message;

    public Hello() {}
    public Hello(int id, String name, String message) {
        super();
        this.id = id;
        this.name = name;
        this.message = message;
    }

    //Setters and Getters

    @Override
    public String toString() {
        return "[" + id + ", " + name + ", " + message + "]";
    }

}
```

## 3. HelloController.java

```
package com.jlcindia.springboot;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.ResponseBody;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
```



```
@Controller
public class HelloController {

    @RequestMapping(value="/myhello", method = RequestMethod.GET)
    @ResponseBody
    public String getMessage1() {
        System.out.println("HelloController - getMessage1()");

        String msg = "Hello , I am getMessage1";

        return msg;
    }

    @RequestMapping(value="/myhello/{name}", method = RequestMethod.GET)
    @ResponseBody
    public String getMessage2(@PathVariable String name) {
        System.out.println("HelloController - getMessage2()");

        String msg = "Hello "+name+" !!! - I am getMessage2";

        return msg;
    }

    @RequestMapping(value="/myhello/name/{name}",method =
    RequestMethod.GET,produces = {"application/json"})
    @ResponseBody
    public Hello getMessage3(@PathVariable String name) {
        System.out.println("HelloController - getMessage3()");

        Hello hello =new Hello(101,name, "I am getMessage3" );

        return hello;
    }

    @RequestMapping(value="/myhello/name/{name}/message/{msg}",
    method = RequestMethod.GET,produces = {"application/xml"})
    @ResponseBody
    public Hello getMessage4(@PathVariable String name,@PathVariable String msg) {
        System.out.println("HelloController - getMessage4()");

        Hello hello =new Hello(109,name, msg);

        return hello;
    }
}
```



```
@RequestMapping(value="/myhello1/name/{name}",
method = RequestMethod.GET, produces = {"application/xml","application/json"})
@ResponseBody
public Hello getMessage5(@PathVariable String name) {
    System.out.println("HelloController - getMessage5()");

    Hello hello =new Hello(12345,name, "I am getMessage5");

    return hello;
}

@RequestMapping(value="/myhello1", method = RequestMethod.GET)
@ResponseBody
public Hello getMessage6(@RequestParam(required = false) String name) {
    System.out.println("HelloController - getMessage6()");

    Hello hello =new Hello(12345,name, "I am getMessage6");

    return hello;
}
}
```

#### 4. application.properties

```
spring.application.name=my-hello-application
server.port=12345
```

#### 5. pom.xml

```
<project ....>

<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.springboot</groupId>
<artifactId>demo1-rest-boot</artifactId>
<version>1.0</version>
<packaging>jar</packaging>

<name>demo1-rest-boot</name>
<description>Spring REST with Boot</description>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.5.3</version>
</parent>
```



```
<dependencies>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-devtools</artifactId>
</dependency>

</dependencies>

<build>
<finalName>myjlcapp</finalName>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>

</plugins>
</build>
</project>
```



## Spring Boot–REST Example

### demo2-rest--boot: Files required

|                           |                             |
|---------------------------|-----------------------------|
| 1. MyBootApplication.java | Same as Demo1               |
| 2. Hello.java             | Same as Demo1               |
| 3. HelloController.java   | <b>Updated in Demo2</b>     |
| 4. JLCAppConfig.java      | <b>Newly Added in Demo2</b> |
| 5. application.properties | Same as Demo1               |
| 6. pom.xml                | Same as Demo1               |

### 3. HelloController.java

```
package com.jlccindia.springboot;
```

```
import org.springframework.web.bind.annotation.GetMapping;  
import org.springframework.web.bind.annotation.PathVariable;  
import org.springframework.web.bind.annotation.RequestParam;  
import org.springframework.web.bind.annotation.RestController;  
/*
```

```
* @Author : Srinivas Dande
```

```
* @company : Java Learning Center
```

```
*/
```

```
@RestController
```

```
public class HelloController {
```

```
    @GetMapping(value = "/myhello")
```

```
    public String getMessage1() {
```

```
        System.out.println("HelloController - getMessage1()");
```

```
        String msg = "Hello , I am getMessage1";
```

```
        return msg;
```

```
    }
```

```
    @GetMapping(value = "/myhello/{name}")
```

```
    public String getMessage2(@PathVariable String name) {
```

```
        System.out.println("HelloController - getMessage2()");
```

```
        String msg = "Hello " + name + " !!! - I am getMessage2";
```

```
        return msg;
```

```
    }
```

```
    @GetMapping(value = "/myhello/name/{name}", produces = { "application/json" })
```

```
    public Hello getMessage3(@PathVariable String name) {
```

```
        System.out.println("HelloController - getMessage3()");
```

```
        Hello hello = new Hello(101, name, "I am getMessage3");
```

```
        return hello;
```

```
    }
```



```
@GetMapping(value = "/myhello/name/{name}/message/{msg}", produces =
{"application/xml" })
public Hello getMessage4(@PathVariable String name, @PathVariable String msg) {
    System.out.println("HelloController - getMessage4()");

    Hello hello = new Hello(109, name, msg);
    return hello;
}

@GetMapping(value = "/myhello1/name/{name}", produces = { "application/xml",
    "application/json" })
public Hello getMessage5(@PathVariable String name) {
    System.out.println("HelloController - getMessage5()");

    Hello hello = new Hello(12345, name, "I am getMessage5");

    return hello;
}

@GetMapping(value = "/myhello1")
public Hello getMessage6(@RequestParam(required = false) String name) {
    System.out.println("HelloController - getMessage6()");

    Hello hello = new Hello(12345, name, "I am getMessage6");

    return hello;
}
}
```

### 3. JLCAppConfig.java

```
package com.jlcindia.springboot;

import java.util.List;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.http.converter.HttpMessageConverter;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@SpringBootApplication
public class JLCAppConfig implements WebMvcConfigurer {
```



```
@Override
public void configureMessageConverters(List<HttpMessageConverter<?>> messageConverters) {

    System.out.println("-----Server Side Converters-----");
    for (HttpMessageConverter<?> myconverter : messageConverters) {
        System.out.println(myconverter);
    }

}
}
```

## demo3-rest-server – REST API Runs here

### demo3-rest-server: Files required

|                           |                         |
|---------------------------|-------------------------|
| 1. MyBootApplication.java | Same as Demo2           |
| 2. Hello.java             | Same as Demo2           |
| 3. HelloController.java   | <b>Updated in Demo3</b> |
| 4. JLCAppConfig.java      | Same as Demo2           |
| 5. application.properties | Same as Demo2           |
| 6. pom.xml                | Same as Demo2           |

### 3. HelloController.java

```
package com.jlccindia.springboot;

import java.io.IOException;
import java.util.Collections;
import java.util.Enumuration;
import java.util.List;

import javax.servlet.http.HttpServletRequest;

import org.springframework.http.HttpHeaders;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;

/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
```



#### **@RestController**

```
public class HelloController {
```

#### **@GetMapping(value="/myhello1")**

```
public Hello getMessage1() throws IOException {  
System.out.println("HelloController - getMessage1() => GET");
```

```
    Hello hello = new Hello(101, "Srinivas Dande", "I am getMessage1" );  
    return hello;  
}
```

#### **@GetMapping(value="/myhello2")**

```
public ResponseEntity<Hello> getMessage2(HttpServletRequest request) throws IOException {  
System.out.println("HelloController - getMessage2() => GET");
```

#### **//Accessing Request Headers**

```
Enumeration<String> enms= request.getHeaderNames();  
List<String> headerNames = Collections.list(enms);
```

```
for(String headerName : headerNames) {  
    System.out.println(headerName+"\t"+ request.getHeader(headerName));  
}
```

#### **//Preparing Response Body**

```
    Hello respBody = new Hello(109, "Srinivas Dande", "I am getMessage2" );
```

#### **//Preparing Response Headers**

```
    HttpHeaders respHeaders = new HttpHeaders();  
    //respHeaders.setContentType(MediaType.APPLICATION_XML);  
    respHeaders.add("my-trainer", "Srinivas Dande");  
    respHeaders.set("my-course", "Spring MicroServices");  
    respHeaders.set("my-access-token", "JLC12345");
```

```
    ResponseEntity<Hello> respEntity =  
    new ResponseEntity<Hello>(respBody, respHeaders, HttpStatus.OK);
```

```
    return respEntity;  
}
```

#### **@PostMapping(value="/myhello1")**

```
public Hello getMessage1(@RequestBody Hello hello) throws IOException {  
System.out.println("HelloController - getMessage1() => POST");
```

```
    hello.setId(12345);  
    hello.setMessage("I am getMessage1() = POST");
```

```
    return hello;  
}
```



```
@PostMapping(value="/myhello2")
```

```
public ResponseEntity<Hello> getMessage2(@RequestBody Hello body,HttpServletRequest  
request) throws IOException {
```

```
System.out.println("HelloController - getMessage2() => POST");
```

```
//Accessing Request Headers
```

```
Enumeration<String> enms= request.getHeaderNames();
```

```
List<String> headerNames = Collections.list(enms);
```

```
for(String headerName : headerNames) {
```

```
System.out.println(headerName+"\t"+ request.getHeader(headerName));
```

```
}
```

```
//Preparing Response Body
```

```
body.setId(12345);
```

```
body.setMessage("I am getMessage1() = POST");
```

```
//Preparing Response Headers
```

```
HttpHeaders respHeaders = new HttpHeaders();
```

```
//respHeaders.setContentType(MediaType.APPLICATION_XML);
```

```
respHeaders.add("my-trainer", "Srinivas Dande");
```

```
respHeaders.set("my-course", "Spring MicroServices");
```

```
respHeaders.set("my-access-token", "JLC12345");
```

```
ResponseEntity<Hello> respEntity =
```

```
new ResponseEntity<Hello>(body,respHeaders, HttpStatus.OK);
```

```
return respEntity;
```

```
}
```

```
}
```

## demo3-rest-client – Calling REST API

### demo3-rest-client: Files required

|                |                      |
|----------------|----------------------|
| 1. Hello.java  | Same as Demo3-Server |
| 2. MyTest.java | New One              |
| 3. Test1.java  | New One              |
| 4. Test2.java  | New One              |
| 5. Test3.java  | New One              |
| 6. Test4.java  | New One              |
| 7. Test5.java  | New One              |
| 8. Test6.java  | New One              |



## 2. MyTest.java

```
package com.jlcindia.springboot;

import java.util.List;
import org.springframework.http.converter.HttpMessageConverter;
import org.springframework.web.client.RestTemplate;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
public class MyTest {
    public static void main(String[] args) {

        RestTemplate restTemp = new RestTemplate();

        System.out.println("-----Client Side Converters-----");
        List<HttpMessageConverter<?>> messageConvertersList =
            restTemp.getMessageConverters();

        for (HttpMessageConverter<?> myconverter : messageConvertersList) {
            System.out.println(myconverter);
        }
    }
}
```

## 3. Test1.java

```
package com.jlcindia.springboot;

import org.springframework.web.client.RestTemplate;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
public class Test1 {
    public static void main(String[] args) {

        RestTemplate restTemp = new RestTemplate();
        String URL="http://localhost:12345/myhello1";

        Hello hello = restTemp.getForObject(URL, Hello.class);
        System.out.println(hello);

    }
}
```



#### **4. Test2.java**

```
package com.jlcindia.springboot;

import org.springframework.http.HttpHeaders;
import org.springframework.http.ResponseEntity;
import org.springframework.web.client.RestTemplate;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
public class Test2 {
    public static void main(String[] args) {

        RestTemplate restTemp = new RestTemplate();
        String URL="http://localhost:12345/myhello1";

        ResponseEntity<Hello> respEntity = restTemp.getForEntity(URL, Hello.class);
        int statusCode = respEntity.getStatusCodeValue();

        System.out.println(statusCode);
        if(statusCode ==200) {
            //Success
            //Access Response Headers
            HttpHeaders respHeaders = respEntity.getHeaders();
            System.out.println(respHeaders);
            String contentType = respHeaders.getContentType().toString();
            System.out.println(contentType);

            //Access Response Body
            Hello hello = (Hello) respEntity.getBody();
            System.out.println(hello);
        }else {
            //Failure
            System.out.println(respEntity.getStatusCode());
        }
    }
}
```



## 5. Test3.java

```
package com.jlcindia.springboot;

import java.util.Collections;

import org.springframework.http.HttpEntity;
import org.springframework.http.HttpHeaders;
import org.springframework.http.HttpMethod;
import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.web.client.RestTemplate;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
public class Test3 {

    public static void main(String[] args) {

        RestTemplate restTemp = new RestTemplate();
        String URL="http://localhost:12345/myhello2";

        //Add Request Headers
        HttpHeaders reqHeaders = new HttpHeaders();
        //reqHeaders.setContentType(MediaType.APPLICATION_XML);
        //reqHeaders.setAccept(Collections.singletonList(MediaType.APPLICATION_XML));

        reqHeaders.setContentType(MediaType.APPLICATION_JSON);
        reqHeaders.setAccept(Collections.singletonList(MediaType.APPLICATION_JSON));

        reqHeaders.add("my-trainer", "Srinivas Dande");
        reqHeaders.set("my-course", "Spring MicroServices");
        reqHeaders.set("payment-status", "none");

        HttpEntity request = new HttpEntity(reqHeaders);

        ResponseEntity<Hello> respEntity = restTemp.exchange(URL,HttpMethod.GET, request,
        Hello.class);

        //Access Response Headers
        HttpHeaders respHeaders = respEntity.getHeaders();
        System.out.println(respHeaders);
        for( String headerName : respHeaders.keySet()) {
            System.out.println(headerName+"\t"+respHeaders.get(headerName));
        }
    }
}
```



#### //Access Response Body

```
Hello hello = (Hello) respEntity.getBody();
System.out.println(hello);
}

}
```

### 6. Test4.java

```
package com.jlcindia.springboot;

import org.springframework.web.client.RestTemplate;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
public class Test4 {
    public static void main(String[] args) {

        RestTemplate restTemp = new RestTemplate();
        String URL="http://localhost:12345/myhello1";

        Hello reqBody = new Hello(0,"Srinivas",null);

        Hello hello = restTemp.postForObject(URL,reqBody, Hello.class);
        System.out.println(hello);

    }
}
```

### 7. Test5.java

```
package com.jlcindia.springboot;

import org.springframework.http.HttpHeaders;
import org.springframework.http.ResponseEntity;
import org.springframework.web.client.RestTemplate;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
public class Test5 {
    public static void main(String[] args) {

        RestTemplate restTemp = new RestTemplate();
        String URL="http://localhost:12345/myhello1";
```



```
Hello reqBody = new Hello(0,"Srinivas",null);

ResponseEntity<Hello> respEntity = restTemp.postForEntity(URL, reqBody,Hello.class);
int statusCode = respEntity.getStatusCodeValue();

System.out.println(statusCode);
if(statusCode ==200) {
//Success
//Access Response Headers
HttpHeaders respHeaders = respEntity.getHeaders();
System.out.println(respHeaders);
String contentType = respHeaders.getContentType().toString();
System.out.println(contentType);

//Access Response Body
Hello hello = (Hello) respEntity.getBody();
System.out.println(hello);

}else {
//Failure
System.out.println(respEntity.getStatusCode());
}

}
}
```

## 8. Test6.java

```
package com.jlcindia.springboot;

import java.util.Collections;
import org.springframework.http.HttpEntity;
import org.springframework.http.HttpHeaders;
import org.springframework.http.HttpMethod;
import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.web.client.RestTemplate;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
public class Test6 {
public static void main(String[] args) {

RestTemplate restTemp = new RestTemplate();
String URL="http://localhost:12345/myhello2";
```



**//Prepare Request Headers**

```
HttpHeaders reqHeaders = new HttpHeaders();
reqHeaders.setContentType(MediaType.APPLICATION_XML);
reqHeaders.setAccept(Collections.singletonList(MediaType.APPLICATION_XML));

//reqHeaders.setContentType(MediaType.APPLICATION_JSON);
//reqHeaders.setAccept(Collections.singletonList(MediaType.APPLICATION_JSON));

reqHeaders.add("my-trainer", "Srinivas Dande");
reqHeaders.set("my-course", "Spring MicroServices");
reqHeaders.set("payment-status", "none");
```

**//Prepare Request Body**

```
Hello reqBody = new Hello(0,"Srinivas",null);

HttpEntity request = new HttpEntity(reqBody,reqHeaders);

ResponseEntity<Hello> respEntity =
restTemp.exchange(URL,HttpMethod.POST, request, Hello.class);
```

**//Access Response Headers**

```
HttpHeaders respHeaders = respEntity.getHeaders();
System.out.println(respHeaders);
for( String headerName : respHeaders.keySet()) {
System.out.println(headerName+"\t"+respHeaders.get(headerName));
}
```

**//Access Response Body**

```
Hello hello = (Hello) respEntity.getBody();
System.out.println(hello);
```

```
}
```

```
}
```



## demo4-rest-server – JLC Bank -REST API Runs

### demo4-rest-server: Files required

|                           |                            |
|---------------------------|----------------------------|
| 1. MyBootApplication.java | 2. AccountController.java  |
| 3. AccountService.java    | 4. AccountServiceImpl.java |
| 5. AccountDAO.java        | 6. Account.java            |
| 7. TxInfo.java            | 8. application.properties  |
| 9. pom.xml                |                            |

#### 1. MyBootApplication.java

```
package com.jlcindia.springboot;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@SpringBootApplication
public class MyBootApplication {
    public static void main(String args[]) {
        SpringApplication.run(MyBootApplication.class, args);
    }
}
```

#### 2. AccountController.java

```
package com.jlcindia.springboot;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@RestController
public class AccountController {
```



@Autowired

AccountService accountService;

**@PostMapping("/addAccount")**

```
public String addAccount(@RequestBody Account acc) {  
    System.out.println("AC-addAccount()");  
    accountService.addAccount(acc);  
    String msg = "Account Added successfully";  
    return msg;  
}
```

**@GetMapping("/myaccount/{accno}")**

```
public Account getAccountByAccno(@PathVariable("accno") int accno) {  
    System.out.println("AC-getAccountByAccno()" + accno);  
    Account acc = accountService.getAccountByAccno(accno);  
    return acc;  
}
```

**@PutMapping("/mydeposit")**

```
public void deposit(@RequestBody TxInfo txInfo) {  
    System.out.println("AC-deposit()");  
    accountService.deposit(txInfo);  
}
```

**@PutMapping("/mywithdraw")**

```
public void withdraw(@RequestBody TxInfo txInfo) {  
    System.out.println("AC-withdraw()");  
    accountService.withdraw(txInfo);  
}
```

**@DeleteMapping("/deleteAccount/{accno}")**

```
public String deleteAccount(@PathVariable int accno) {  
    System.out.println("AC-deleteAccount()");  
    accountService.deleteAccount(accno);  
    String msg = "Account Deleted successfully";  
    return msg;  
}
```

**@GetMapping("/myaccounts")**

```
public List<Account> getAllAccounts() {  
    System.out.println("AC-getAllAccounts()");  
    List<Account> accList = accountService.getAllAccounts();  
    return accList;  
}
```



#### **@GetMapping("/myaccounts/city/{city}")**

```
public List<Account> getAccountsByCity(@PathVariable String city) {  
    System.out.println("AC-getAllAccounts()");  
    List<Account> accList = accountService.getAccountsByCity(city);  
    return accList;  
}
```

#### **@GetMapping("/myaccounts/bcode/{bcode}")**

```
public List<Account> getAccountsByBcode(@PathVariable String bcode) {  
    System.out.println("AC-getAllAccounts()");  
    List<Account> accList = accountService.getAccountsByBcode(bcode);  
    return accList;  
}  
}
```

### **3. AccountService.java**

```
package com.jlcindia.springboot;  
  
import java.util.List;  
/*  
 * @Author : Srinivas Dande  
 * @company : Java Learning Center  
 */  
public interface AccountService {  
    public void deposit(TxInfo txInfo);  
    public void withdraw(TxInfo txInfo);  
    public void deleteAccount(int accno);  
    public void addAccount(Account acc);  
  
    public Account getAccountByAccno(int accno);  
    public List<Account> getAllAccounts();  
    public List<Account> getAccountsByCity(String city);  
    public List<Account> getAccountsByBcode(String bcode);  
}
```

### **4. AccountServiceImpl.java**

```
package com.jlcindia.springboot;  
  
import java.util.List;  
import java.util.Optional;  
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.stereotype.Service;  
import org.springframework.transaction.annotation.Transactional;  
/*  
 * @Author : Srinivas Dande  
 * @company : Java Learning Center  
 */
```



**@Service**

**@Transactional**

```
public class AccountServiceImpl implements AccountService {
```

**@Autowired**

```
AccountDAO accountDAO;
```

**@Override**

```
public void addAccount(Account acc) {  
    System.out.println("ADAO-addAccount()");  
    accountDAO.save(acc);  
}
```

**@Override**

```
public Account getAccountByAccno(int accno) {  
    System.out.println("ADAO-getAccountByAccno()");  
    Optional<Account> opt=accountDAO.findById(accno);  
    Account acc=null;  
    if(opt.isPresent()) {  
        acc=opt.get();  
    }  
    return acc;  
}
```

**@Override**

```
public void deposit(TxInfo txInfo) {  
    System.out.println("ADAO-deposit()");  
    Optional<Account> opt=accountDAO.findById(txInfo.getAccno());  
    if(opt.isPresent()) {  
        Account acc=opt.get();  
        double updatedBal=acc.getBalance()+txInfo.getAmount();  
        acc.setBalance(updatedBal);  
        accountDAO.save(acc);  
    }  
}
```

**@Override**

```
public void withdraw(TxInfo txInfo) {  
    System.out.println("ADAO-withdraw()");  
    Optional<Account> opt=accountDAO.findById(txInfo.getAccno());  
    if(opt.isPresent()) {  
        Account acc=opt.get();  
        double updatedBal=acc.getBalance()-txInfo.getAmount();  
        acc.setBalance(updatedBal);  
        accountDAO.save(acc);  
    }  
}
```



```
@Override
public void deleteAccount(int accno) {
    System.out.println("ADAO-deleteAccount()");
    accountDAO.deleteById(accno);
}

@Override
public List<Account> getAllAccounts() {
    System.out.println("ADAO-getAllAccounts()");
    return accountDAO.findAll();
}

@Override
public List<Account> getAccountsByCity(String city) {
    System.out.println("ADAO-getAccountsByCity()");
    return accountDAO.getAccountsByCity(city);
}

@Override
public List<Account> getAccountsByBcode(String bcode) {
    System.out.println("ADAO-getAccountsByBcode()");
    return accountDAO.getAccountsByBcode(bcode);
}
}
```

### **5. AccountDAO.java**

```
package com.jlcindia.springboot;

import java.util.List;

import org.springframework.data.jpa.repository.JpaRepository;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

public interface AccountDAO extends JpaRepository<Account, Integer> {
    public List<Account> getAccountsByCity(String city);
    public List<Account> getAccountsByBcode(String bcode);
}
```



## 6. Account.java

```
package com.jlcindia.springboot;

import javax.persistence.*;

@Entity
@Table(name = "myaccounts")
public class Account {

    @Id
    @Column(name = "accno")
    @GeneratedValue(strategy = GenerationType.AUTO)
    private int accno;

    @Column(name = "atype")
    private String atype;

    @Column(name = "bcode")
    private String bcode;

    @Column(name = "balance")
    private double balance;

    @Column(name = "city")
    private String city;

    public Account() {}
    public Account( String atype, String bcode, double balance, String city) {
        super();
        this.atype = atype;
        this.bcode = bcode;
        this.balance = balance;
        this.city = city;
    }

    public Account(int accno, String atype, String bcode, double balance, String city) {
        super();
        this.accno = accno;
        this.atype = atype;
        this.bcode = bcode;
        this.balance = balance;
        this.city = city;
    }
    //Setters and Getters
    //toString()
}
```



### 7. TxInfo.java

```
package com.jlcindia.springboot;

public class TxInfo {
    private int accno;
    private double amount;

    public TxInfo() {}
    public TxInfo(int accno, double amount) {
        super();
        this.accno = accno;
        this.amount = amount;
    }
    //Setters and Getters
}
```

### 8. application.properties

```
spring.application.name=my-jlc-bank-app
server.port=12345

spring.datasource.driver-class-name=com.mysql.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/myjlcdb5
spring.datasource.username=root
spring.datasource.password=srinivas

spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update
```

### 9. pom.xml

```
<project ...>

<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.springboot</groupId>
<artifactId>demo4-rest-server</artifactId>
<version>1.0</version>
<packaging>jar</packaging>
<name>demo4-rest-server</name>
<description>Spring REST and Data JPA with Boot</description>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.5.3</version>
</parent>
```



```
<dependencies>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<dependency>
<groupId>mysql</groupId>
<artifactId>mysql-connector-java</artifactId>
<version>5.1.32</version>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-devtools</artifactId>
</dependency>

</dependencies>

<build>
<finalName>myjlcapp</finalName>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>

</plugins>
</build>
</project>
```

## demo4-rest-client – Calling REST API

### demo4-rest-client: Files required

|                 |                |
|-----------------|----------------|
| 1. Account.java | 2. TxInfo.java |
| 3. Test1.java   | 4. Test2.java  |
| 5. Test3.java   | 6. Test4.java  |
| 7. Test5.java   | 8. Test6.java  |
| 9. Test7.java   |                |



### 1. Account.java

```
package com.jlcindia.springboot;

/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

public class Account {

    private int accno;
    private String atype;
    private String bcode;
    private double balance;
    private String city;

    //Constrcutors
    //Setters and Getters
    //toString()
}
```

### 2. TxInfo.java

```
package com.jlcindia.springboot;

/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

public class TxInfo {
    private int accno;
    private double amount;

    //Constrcutors
    //Setters and Getters

}
```



### 3. Test1.java

```
package com.jlcindia.springboot;

import org.springframework.web.client.RestTemplate;

/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
public class Test1 {
    public static void main(String[] args) {

        RestTemplate restTemp=new RestTemplate();
        String URL="http://localhost:12345/addAccount";

        Account acc=new Account("SA","B-99",75000,"Hyd");
        String msg=restTemp.postForObject(URL, acc, String.class);
        System.out.println(msg);

        System.out.println("Done");
    }
}
```

### 4. Test2.java

```
package com.jlcindia.springboot;

import org.springframework.web.client.RestTemplate;

/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
public class Test2 {
    public static void main(String[] args) {

        RestTemplate restTemp=new RestTemplate();
        String URL="http://localhost:12345/myaccount/101";

        Account acc=restTemp.getForObject(URL, Account.class) ;

        System.out.println(acc);
        System.out.println("Done");

    }
}
```



### 5. Test3.java

```
package com.jlcindia.springboot;

import org.springframework.web.client.RestTemplate;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
public class Test3 {
    public static void main(String[] args) {

        RestTemplate restTemp=new RestTemplate();
        String URL="http://localhost:12345/mydeposit";

        TxInfo txInfo=new TxInfo(101, 5000);
        restTemp.put(URL,txInfo) ;

        System.out.println("Done");

    }
}
```

### 6. Test4.java

```
package com.jlcindia.springboot;

import org.springframework.web.client.RestTemplate;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
public class Test4 {
    public static void main(String[] args) {

        RestTemplate restTemp=new RestTemplate();
        String URL="http://localhost:12345/mywithdraw";

        TxInfo txInfo=new TxInfo(102, 10000);

        restTemp.put(URL,txInfo) ;
        System.out.println("Done");

    }
}
```



### 7. Test5.java

```
package com.jlcindia.springboot;

import org.springframework.web.client.RestTemplate;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

public class Test5 {
    public static void main(String[] args) {
        RestTemplate restTemp = new RestTemplate();
        String URL = "http://localhost:54321/deleteAccount/5";

        restTemp.delete(URL);

        System.out.println("Done");
    }
}
```

### 8. Test6.java

```
package com.jlcindia.springboot;

import java.util.*;
import org.springframework.web.client.RestTemplate;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

public class Test6 {
    public static void main(String[] args) {

        RestTemplate restTemp = new RestTemplate();
        String URL = "http://localhost:54321/myaccounts";

        List<Map<String, String>> accList = restTemp.getForObject(URL, List.class);
        for (Map<String, String> mymap : accList) {
            System.out.println(mymap);
        }

        System.out.println("Done");
    }
}
```



### 9. Test7.java

```
package com.jlcindia.springboot;

import java.util.List;
import java.util.Map;

import org.springframework.http.ResponseEntity;
import org.springframework.web.client.RestTemplate;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

public class Test7 {
    public static void main(String[] args) {

        RestTemplate restTemp = new RestTemplate();
        String URL = "http://localhost:54321/myaccounts/city/Blore";

        ResponseEntity<List> respEntity = restTemp.getForEntity(URL, List.class);

        List<Map<String, String>> accList = respEntity.getBody();
        for (Map<String, String> mymap : accList) {
            System.out.println(mymap);
        }

        System.out.println("Done");
    }
}
```



## demo5-rest-server – Handling Exceptions Locally

### demo5-rest-server: Files required

|                                     |                                     |
|-------------------------------------|-------------------------------------|
| 1. MyBootApplication.java           | 2. Customer.java                    |
| 3. CustomerController.java          | 4. CustomerService.java             |
| 5. CustomerIDNotFoundException.java | 6. InvalidPhoneNumberException.java |
| 7. CityNotFoundException.java       | 8. MyErrorMessage.java              |
| 9. application.properties           | 10. pom.xml                         |

#### 1. MyBootApplication.java

```
package com.jlclndia.springboot;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@SpringBootApplication
public class MyBootApplication {
    public static void main(String args[]) {
        SpringApplication.run(MyBootApplication.class, args);
    }
}
```

#### 2. Customer.java

```
package com.jlclndia.springboot;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
public class Customer {
    private int cid;
    private String cname;
    private String email;
    private long phone;
    private String city;

    //Constrcutors
    //Setters and Getters

    @Override
    public String toString() {
        return "Customer [" + cid + ", " + cname + ", " + email + ", " + phone + ", " + city + "];"
    }
}
```



### 3. CustomerController.java

```
package com.jlcindia.springboot;

import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpHeaders;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.server.ResponseStatusException;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

@RestController
public class CustomerController {

    @Autowired
    CustomerService customerService;

    @GetMapping("/mycustomer/custId/{custId}")
    public Customer getCustomerByCid(@PathVariable("custId") int custId) {
        System.out.println("CC-getCustomerByCid() - " + custId);

        Customer cust = null;

        try {
            cust = customerService.getCustomerByCid(custId);
        } catch (CustomerIDNotFoundException ex) {
            throw new ResponseStatusException(HttpStatus.NOT_FOUND, "MYCustomer ID Not Found", ex);
        }

        return cust;
    }

    @GetMapping(value = "/mycustomer/phone/{phone}")
    public ResponseEntity<?> getCustomerByPhone(@PathVariable("phone") long phone) {
        System.out.println("CC-getCustomerByPhone() - " + phone);

        HttpHeaders respHeaders = new HttpHeaders();
        Customer cust = null;
```



```
try {
    cust = customerService.getCustomerByPhone(phone);
    respHeaders.add("message", "Phone Number Found");
    return new ResponseEntity<>(cust, respHeaders, HttpStatus.OK);

} catch (InvalidPhoneNumberException ex) {
    respHeaders.add("message", "Phone Number Not Found");
}

MyErrorMessage errMsg = new MyErrorMessage();
errMsg.setMytimestamp(LocalDateTime.now());
errMsg.setMyerror(" Phone Number Not Found");
errMsg.setMystatus(HttpStatus.FORBIDDEN.value());

return new ResponseEntity<>(errMsg, respHeaders, HttpStatus.FORBIDDEN);

}

@GetMapping(value = "/mycustomer/city/{city}")
public ResponseEntity<?> getCustomersByCity(@PathVariable("city") String city) {
    System.out.println("CC-getCustomersByCity() - " + city);

    HttpHeaders respHeaders = new HttpHeaders();
    List<Customer> custList = new ArrayList<>();

    try {
        custList = customerService.getCustomersByCity(city);
        respHeaders.add("message", "Customers Found");
        return new ResponseEntity<>(custList, respHeaders, HttpStatus.OK);
    } catch (CityNotFoundException ex) {
        respHeaders.add("message", "Customers Not Found");
    }

    MyErrorMessage errMsg = new MyErrorMessage();
    errMsg.setMytimestamp(LocalDateTime.now());
    errMsg.setMyerror(" No Customers with given City");
    errMsg.setMystatus(HttpStatus.BAD_REQUEST.value());

    return new ResponseEntity<>(errMsg, HttpStatus.BAD_REQUEST);
}

}
```



#### 4. CustomerService.java

```
package com.jlcindia.springboot;

import java.util.*;
import org.springframework.stereotype.Service;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@Service
public class CustomerService {

    public Customer getCustomerByCid(int custId) {
        Customer cust = null;
        if (custId >= 100) {
            cust = new Customer(custId, "Srinivas", "sri@jlc.com", 12345, "Blore");
        } else {
            throw new CustomerIDNotFoundException();
        }
        return cust;
    }

    public Customer getCustomerByPhone(long phone) {
        Customer cust = null;
        if (phone >= 50000) {
            cust = new Customer(109, "Dande", "dande@jlc.com", phone, "Blore");
        } else {
            throw new InvalidPhoneNumberException();
        }
        return cust;
    }

    public List<Customer> getCustomersByCity(String city) {
        List<Customer> custList = new ArrayList<>();
        if (city.equals("Blore")) {
            custList.add(new Customer(101, "Sri", "sri@jlc.com", 111, city));
            custList.add(new Customer(102, "Vas", "vas@jlc.com", 222, city));
            custList.add(new Customer(103, "SD", "sd@jlc.com", 333, city));
            custList.add(new Customer(104, "DS", "ds@jlc.com", 444, city));
            custList.add(new Customer(105, "Dande", "dande@jlc.com", 555, city));
            custList.add(new Customer(106, "Dande", "dande@jlc.com", 555, city));
        } else {
            throw new CityNotFoundException();
        }
        return custList;
    }
}
```



#### 5. CustomerIDNotFoundException.java

```
package com.jlcindia.springboot;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

public class CustomerIDNotFoundException extends RuntimeException{

}
```

#### 6. InvalidPhoneNumberException.java

```
package com.jlcindia.springboot;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

public class InvalidPhoneNumberException extends RuntimeException{

}
```

#### 7. CityNotFoundException.java

```
package com.jlcindia.springboot;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

public class CityNotFoundException extends RuntimeException {

}
```

#### 8. MyErrorMessage.java

```
package com.jlcindia.springboot;

import com.fasterxml.jackson.annotation.JsonFormat;
import java.time.LocalDateTime;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

public class MyErrorMessage {
```



```
@JsonFormat(shape = JsonFormat.Shape.STRING, pattern = "yyyy-MM-dd hh:mm:ss")
private LocalDateTime mytimestamp;
private int mystatus;
private String myerror;

public MyErrorMessage() {
}

//Constructors
//Setters and Getters

}
```

### 9. application.properties

```
spring.application.name=my-jlc-app
server.port=12345
```

### 10. pom.xml

```
<project ...>
<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.springboot</groupId>
<artifactId>demo5-rest-server</artifactId>
<version>1.0</version>
<packaging>jar</packaging>
<name>demo5-rest-server</name>
<description>Exception Handling- Locally</description>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.5.3</version>
</parent>

<dependencies>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-devtools</artifactId>
</dependency>

</dependencies>
```



```
<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>

</plugins>
</build>
</project>
```

## demo5-rest-client – Calling REST API

### demo5-rest-client: Files required

|                  |               |
|------------------|---------------|
| 1. Customer.java | 2. Test1.java |
| 3. Test2.java    | 4. Test3.java |

#### 1. Customer.java

```
package com.jlcindia.springboot;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
public class Customer {
private int cid;
private String cname;
private String email;
private long phone;
private String city;

//Constrcutors
//Setters and Getters

@Override
public String toString() {
return "Customer [" + cid + ", " + cname + ", " + email + ", " + phone + ", " + city + "];"
}
}
```



## 2. Test1.java

```
package com.jlcindia.springboot;

import org.springframework.http.HttpEntity;
import org.springframework.http.HttpMethod;
import org.springframework.http.ResponseEntity;
import org.springframework.web.client.RestTemplate;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

public class Test1 {

    public static void main(String[] args) {

        RestTemplate restTemp = new RestTemplate();

        String URL1 = "http://localhost:54321/mycustomer/custId/123"; //Success
        String URL2 = "http://localhost:54321/mycustomer/custId/12"; //Failure

        ResponseEntity<Customer> respEntity=null;
        try {
            respEntity = restTemp.exchange(URL2,HttpMethod.GET, HttpEntity.EMPTY, Customer.class);

            Customer cust = respEntity.getBody();
            System.out.println(cust);

        }catch(Exception ex) {
            System.out.println("1."+ respEntity);
            System.out.println("2."+ ex.getMessage());
            System.out.println("3.-----Customer Id Not Found-----");
        }

        System.out.println("Done");

    }

}
```



#### 4. Test2.java

```
package com.jlcindia.springboot;

import org.springframework.http.HttpEntity;
import org.springframework.http.HttpMethod;
import org.springframework.http.ResponseEntity;
import org.springframework.web.client.RestTemplate;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

public class Test2 {

    public static void main(String[] args) {

        RestTemplate restTemp = new RestTemplate();

        String URL1 = "http://localhost:54321/mycustomer/phone/54321"; //Success
        String URL2 = "http://localhost:54321/mycustomer/phone/12345"; //Failure

        ResponseEntity<Customer> respEntity=null;
        try {
            respEntity = restTemp.exchange(URL2,HttpMethod.GET, HttpEntity.EMPTY, Customer.class);

            Customer cust = respEntity.getBody();
            System.out.println(cust);

        }catch(Exception ex) {
            System.out.println("1."+ respEntity);
            System.out.println("2."+ ex.getMessage());
            System.out.println("3.-----Customer Phone Not Available-----");
        }

        System.out.println("Done");

    }

}
```



## 5. Test3.java

```
package com.jlcindia.springboot;

import java.util.List;
import java.util.Map;

import org.springframework.http.HttpEntity;
import org.springframework.http.HttpMethod;
import org.springframework.http.ResponseEntity;
import org.springframework.web.client.RestTemplate;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

public class Test3 {

    public static void main(String[] args) {

        RestTemplate restTemp = new RestTemplate();

        String URL1 = "http://localhost:54321/mycustomer/city/Blore"; // Success
        String URL2 = "http://localhost:54321/mycustomer/city/Hyd"; // Failure

        ResponseEntity<List> respEntity=null;
        try {
            respEntity = restTemp.exchange(URL2,HttpMethod.GET, HttpEntity.EMPTY, List.class);

            List<Map<String,String>> mylist = respEntity.getBody() ;
            if (!mylist.isEmpty()) {
                for(Map mymap : mylist)
                    System.out.println(mymap);
            }

        }catch(Exception ex) {
            System.out.println("1."+ respEntity);
            System.out.println("2."+ ex.getMessage());
            System.out.println("3.-----No Customers with Given City-----");
        }

        System.out.println("Done");

    }

}
```



## demo6-rest-server – Handling Exceptions Globally

### demo6-rest-server: Files required

|                                     |                             |
|-------------------------------------|-----------------------------|
| 1. MyBootApplication.java           | Same as Demo5               |
| 2. Customer.java                    | Same as Demo5               |
| 3. CustomerController.java          | <b>Updated in Demo6</b>     |
| 4. MyExceptionHandler.java          | <b>Newly Added in Demo6</b> |
| 5. CustomerService.java             | Same as Demo5               |
| 6. CustomerIDNotFoundException.java | Same as Demo5               |
| 7. InvalidPhoneNumberException.java | Same as Demo5               |
| 8. CityNotFoundException.java       | Same as Demo5               |
| 9. MyErrorMessage.java              | Same as Demo5               |
| 10. application.properties          | Same as Demo5               |
| 11. pom.xml                         | Same as Demo5               |

### 3. CustomerController.java

```
package com.jlcindia.springboot;

import java.util.ArrayList;
import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpHeaders;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@RestController
public class CustomerController {

    @Autowired
    CustomerService customerService;

    @GetMapping("/mycustomer/custId/{custId}")
    public Customer getCustomerByCid(@PathVariable("custId") int custId) {
        System.out.println("CC-getCustomerByCid() - " + custId);

        Customer cust = customerService.getCustomerByCid(custId);
        return cust;
    }
}
```



```
@GetMapping(value = "/mycustomer/phone/{phone}")
public ResponseEntity<Customer> getCustomerByPhone(@PathVariable("phone") long phone) {
    System.out.println("CC-getCustomerByPhone() - " + phone);

    Customer cust = customerService.getCustomerByPhone(phone);
    ResponseEntity<Customer> respEntity =
    new ResponseEntity<Customer>(cust ,HttpHeaders.EMPTY,HttpStatus.OK);

    return respEntity;
}

@GetMapping(value = "/mycustomer/city/{city}")
public ResponseEntity<List<Customer>> getCustomersByCity(@PathVariable("city") String city) {
    System.out.println("CC-getCustomersByCity() - " + city);

    List<Customer> custList =new ArrayList<>();

    custList = customerService.getCustomersByCity(city);

    ResponseEntity<List<Customer>> respEntity =
    new ResponseEntity<List<Customer>>(custList ,HttpHeaders.EMPTY,HttpStatus.OK);

    return respEntity;
}
}
```

#### **4. MyExceptionHandler.java**

```
package com.jlcindia.springboot;

import java.time.LocalDateTime;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ControllerAdvice;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.context.request.WebRequest;
import org.springframework.web.servlet.mvc.method.annotation.ResponseEntityExceptionHandler;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
```



#### @ControllerAdvice

```
public class MyExceptionHandler extends ResponseEntityExceptionHandler {
```

```
    @ExceptionHandler({ CustomerIDNotFoundException.class })
```

```
    public ResponseEntity<MyErrorMessage> handleCidNotFound() {
```

```
        MyErrorMessage errMsg = new MyErrorMessage();
        errMsg.setMytimestamp(LocalDateTime.now());
        errMsg.setMyerror("Customer Id is Not Available");
        errMsg.setMystatus(HttpStatus.NOT_FOUND.value());
```

```
        return new ResponseEntity<>(errMsg, HttpStatus.NOT_FOUND);
    }
```

```
    @ExceptionHandler({ InvalidPhoneNumberException.class })
```

```
    public ResponseEntity<MyErrorMessage> handlePhoneNotFound(RuntimeException ex,
        WebRequest request) {
```

```
        MyErrorMessage errMsg = new MyErrorMessage();
        errMsg.setMytimestamp(LocalDateTime.now());
        errMsg.setMyerror(" Phone Number Not Found");
        errMsg.setMystatus(HttpStatus.FORBIDDEN.value());
```

```
        return new ResponseEntity<>(errMsg, HttpStatus.FORBIDDEN);
    }
```

```
    @ExceptionHandler({ CityNotFoundException.class })
```

```
    public ResponseEntity<MyErrorMessage> handleCityNotFound(RuntimeException ex, WebRequest
        request) {
```

```
        MyErrorMessage errMsg = new MyErrorMessage();
        errMsg.setMytimestamp(LocalDateTime.now());
        errMsg.setMyerror(" No Customers with given City");
        errMsg.setMystatus(HttpStatus.BAD_REQUEST.value());
```

```
        return new ResponseEntity<>(errMsg, HttpStatus.BAD_REQUEST);
    }
}
```

### demo6-rest-client – Calling REST API

#### demo6-rest-client: Files required

|                  |               |
|------------------|---------------|
| 1. Customer.java | Same as Demo5 |
| 2. Test1.java    | Same as Demo5 |
| 3. Test2.java    | Same as Demo5 |
| 4. Test3.java    | Same as Demo5 |



## demo7-rest-swagger – Swagger Example

### A) Develop the REST API

- 1) Copy demo1-rest-boot as demo7-rest-swagger
- 2) Delete the following
  - a. Hello.java
  - b. HelloController.java
- 3) Update pom.xml
- 4) Update application.properties

### 5) Write Account.java

```
public class Account {  
  
    private int accno;  
    private String atype;  
    private String bcode;  
    private double balance;  
  
    //Constrcutors  
    //Setters and Getters  
    //toString()  
}
```

### 6) Write AccountController.java

```
package com.jlcindia.springboot;  
  
import java.util.*;  
import org.springframework.web.bind.annotation.*;  
  
@RestController  
public class AccountController {  
  
    @GetMapping("/mybal/{accno}")  
    public double getBalance(@PathVariable("accno") String accno) {  
        System.out.println("AC-getBalance()" + accno);  
        return 25000;  
    }  
}
```



```
@GetMapping("/myaccount/{accno}")
public Account getAccountByAccno(@PathVariable("accno") int accno) {
    System.out.println("AC-getAccountByAccno()" + accno);

    Account acc = new Account(accno, "SA", "B99", 29000.0);
    return acc;
}

@GetMapping("/myaccounts")
public List<Account> getAllCounts() {
    System.out.println("AC-getAllCounts()");

    List<Account> accList = new ArrayList<>();
    accList.add(new Account(101, "SA", "B99", 19000.0));
    accList.add(new Account(102, "SA", "B99", 29000.0));
    accList.add(new Account(103, "SA", "B99", 29000.0));
    return accList;
}

@PostMapping(value = "/addAccount")
public Account addAccount(@RequestBody Account acc) {
    System.out.println("AC-addAccount()");
    acc.setAccno(123);
    return acc;
}

@PutMapping(value = "/updateAccount")
public String updateAccount(@RequestBody Account acc) {
    System.out.println("AC-updateAccount()");
    String msg = "Account Updated Successfully";
    return msg;
}

@DeleteMapping(value = "/deleteAccount/{myaccno}")
public String deleteAccount(@PathVariable("myaccno") String accno) {
    System.out.println("AC-deleteAccount()");
    String msg = "Account Deleted Successfully";
    return msg;
}
}
```



## 7) Write Customer.java

```
package com.jlcindia.springboot;

public class Customer {
    private int cid;
    private String cname;
    private String email;
    private long phone;
    private String city;

    //Constrcutors
    //Setters and Getters
    //toString()

}
```

## 8) Write CustomerController.java

-----

```
package com.jlcindia.springboot;

import java.util.*;
import org.springframework.web.bind.annotation.*;

@RestController
public class CustomerController {

    @GetMapping(value = "/mycustomer/custId/{custId}", produces =
"application/xml")
    public Customer getCustomerByCid(@PathVariable("custId") int custId) {
        System.out.println("CC-getCustomerByCid() - " + custId);
        Customer cust = new Customer(custId, "Srinivas", "sri@jlc.com", 12345, "Blore");
        return cust;
    }

    @GetMapping(value = "/mycustomer/phone/{phone}", produces =
"application/xml")
    public Customer getCustomerByPhone(@PathVariable("phone") long phone) {
        System.out.println("CC-getCustomerByPhone() - " + phone);

        Customer cust = new Customer(109, "Dande", "dande@jlc.com", phone, "Blore");
        return cust;
    }
}
```



```
@GetMapping(value = "/mycustomer/city/{city}", produces = "application/json")
public List<Customer> getCustomersByCity(@PathVariable("city") String city) {
    System.out.println("CC-getCustomersByCity() - " + city);
    List<Customer> custList = new ArrayList<>();
    custList.add(new Customer(101, "Sri", "sri@jlc.com", 111, city));
    custList.add(new Customer(102, "Vas", "vas@jlc.com", 222, city));
    custList.add(new Customer(103, "SD", "sd@jlc.com", 333, city));
    return custList;
}

}
```

---

## **B)Add Swagger to the Project**

### **9) Add the following dependencies in pom.xml**

---

```
<dependency>
<groupId>io.springfox</groupId>
<artifactId>springfox-swagger2</artifactId>
<version>2.4.0</version>
</dependency>

<dependency>
<groupId>io.springfox</groupId>
<artifactId>springfox-swagger-ui</artifactId>
<version>2.4.0</version>
</dependency>
```

### **10) Mark the Boot main class with @EnableSwagger2 annotation**

---

```
@SpringBootApplication
@EnableSwagger2
public class MyBootApplication{
    public static void main(String args[]) {
        SpringApplication.run(StartMyBootApplication.class, args);
    }
}
```

11) Add the Required Beans Bean Configuration Class.

12) Mark the Documentaion Related Annotations in POJO Classes/Resource Classes.

```
@ApiModel
@ApiModelProperty
```

13) Mark the Documentaion Related Annotations in Controllers.

```
@ApiOperation
```

14) Start the Application

15) pen the following URL

```
http://localhost:12345/swagger-ui.html
```

### demo7-rest-swagger – Swagger Example

#### demo7-rest-swagger: Files required

|                            |                              |
|----------------------------|------------------------------|
| 1. MyBootApplication.java  | 2. Account.java              |
| 3. AccountController.java  | 4. Customer.java             |
| 5. CustomerController.java | 6. JLCApplicationConfig.java |
| 7. application.properties  | 8. pom.xml                   |

#### 1. MyBootApplication.java

```
package com.jlccindia.springboot;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

import springfox.documentation.swagger2.annotations.EnableSwagger2;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

@SpringBootApplication
@EnableSwagger2
public class MyBootApplication {
    public static void main(String as[]) {
        SpringApplication.run(MyBootApplication.class, as);
    }
}
```



## 2. Account.java

```
package com.jlcindia.springboot;

import io.swagger.annotations.ApiModel;
import io.swagger.annotations.ApiModelProperty;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

@ApiModel(value="Account contains Accno,AccType,BranchCode,balance ")
public class Account {

    @ApiModelProperty(value="Contains Acc Number")
    private int accno;

    @ApiModelProperty(value="Contains Acc Type")
    private String atype;

    @ApiModelProperty(value="Contains Branch")
    private String bcode;

    @ApiModelProperty(value="Contains Acc Balance")
    private double balance;

    //Constrcutors
    //Setters and Getters
    //toString()
}
```

## 3. AccountController.java

```
package com.jlcindia.springboot;

import java.util.*;
import org.springframework.web.bind.annotation.*;
import io.swagger.annotations.ApiOperation;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@RestController
public class AccountController {

    @ApiOperation(value = " get the Balance", response = double.class, notes = "Gives Balance of Customer with Accno")
    @GetMapping("/mybal/{accno}")
    public double getBalance(@PathVariable("accno") String accno) {
```



```
System.out.println("AC-getBalance()" + accno);  
return 25000;
```

```
}
```

```
@ApiOperation(value = " get the Account Information", response = Account.class, notes =  
"Gives Complete Account Information with Accno")
```

```
@GetMapping("/myaccount/{accno}")
```

```
public Account getAccountByAccno(@PathVariable("accno") int accno) {  
System.out.println("AC-getAccountByAccno()" + accno);
```

```
Account acc = new Account(accno, "SA", "B99", 29000.0);  
return acc;
```

```
}
```

```
@ApiOperation(value = " get All Accounts Information", response = List.class, notes = "Gives  
All the Accounts Information")
```

```
@GetMapping("/myaccounts")
```

```
public List<Account> getAllCounts() {  
System.out.println("AC-getAllCounts()");
```

```
List<Account> accList = new ArrayList<>();  
accList.add(new Account(101, "SA", "B99", 19000.0));  
accList.add(new Account(102, "SA", "B99", 29000.0));  
accList.add(new Account(103, "SA", "B99", 29000.0));  
return accList;  
}
```

```
@ApiOperation(value = " Adds Account Info", response = Account.class, notes = "Adds  
Accounts Information")
```

```
@PostMapping(value = "/addAccount")
```

```
public Account addAccount(@RequestBody Account acc) {  
System.out.println("AC-addAccount()");
```

```
acc.setAccno(123);  
return acc;  
}
```

```
@ApiOperation(value = " Updates Account Info", response = String.class, notes = "Updates  
Accounts Information")
```

```
@PutMapping(value = "/updateAccount")
```

```
public String updateAccount(@RequestBody Account acc) {  
System.out.println("AC-updateAccount()");
```

```
String msg = "Account Updated Successfully";  
return msg;  
}
```



```
@ApiOperation(value = " Delete Account Info", response = String.class, notes = "Delete Accounts Information")
```

```
@DeleteMapping(value = "/deleteAccount/{myaccno}")
```

```
public String deleteAccount(@PathVariable("myaccno") String accno) {  
    System.out.println("AC-deleteAccount()");
```

```
    String msg = "Account Deleted Successfully";  
    return msg;
```

```
}
```

```
}
```

#### 4. Customer.java

```
package com.jlcindia.springboot;
```

```
import javax.xml.bind.annotation.XmlRootElement;
```

```
import io.swagger.annotations.ApiModel;
```

```
import io.swagger.annotations.ApiModelProperty;
```

```
/*
```

```
 * @Author : Srinivas Dande
```

```
 * @company : Java Learning Center
```

```
 */
```

```
@ApiModel(value = "Customer contains cid,Cname,email,phone,city ")
```

```
@XmlRootElement
```

```
public class Customer {
```

```
    @ApiModelProperty(value = "Contains Cust Id")
```

```
    private int cid;
```

```
    @ApiModelProperty(value = "Contains Cust Name")
```

```
    private String cname;
```

```
    @ApiModelProperty(value = "Contains Email")
```

```
    private String email;
```

```
    @ApiModelProperty(value = "Contains Phone")
```

```
    private long phone;
```

```
    @ApiModelProperty(value = "Contains City")
```

```
    private String city;
```

```
    //Constrcutors
```

```
    //Setters and Getters
```

```
    //toString()
```

```
}
```



## 5. CustomerController.java

```
package com.jlcindia.springboot;

import java.util.*;
import org.springframework.web.bind.annotation.*;
import io.swagger.annotations.ApiOperation;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@RestController
public class CustomerController {

    @ApiOperation(value = " get the Customer By Cid", response = double.class, notes = "Gives Customer Full Information By CID")
    @GetMapping(value = "/mycustomer/custId/{custId}", produces = "application/xml")
    public Customer getCustomerByCid(@PathVariable("custId") int custId) {
        System.out.println("CC-getCustomerByCid() - " + custId);

        Customer cust = new Customer(custId, "Srinivas", "sri@jlc.com", 12345, "Blore");
        return cust;
    }

    @ApiOperation(value = " get the Customer By Phone", response = Customer.class, notes = "Gives Customer Full Information By Phone")
    @GetMapping(value = "/mycustomer/phone/{phone}", produces = "application/xml")
    public Customer getCustomerByPhone(@PathVariable("phone") long phone) {
        System.out.println("CC-getCustomerByPhone() - " + phone);

        Customer cust = new Customer(109, "Dande", "dande@jlc.com", phone, "Blore");
        return cust;
    }

    @ApiOperation(value = " get the Customer By City", response = List.class, notes = "Gives Customers Full Information By City")
    @GetMapping(value = "/mycustomer/city/{city}", produces = "application/json")
    public List<Customer> getCustomersByCity(@PathVariable("city") String city) {
        System.out.println("CC-getCustomersByCity() - " + city);

        List<Customer> custList = new ArrayList<>();
        custList.add(new Customer(101, "Sri", "sri@jlc.com", 111, city));
        custList.add(new Customer(102, "Vas", "vas@jlc.com", 222, city));
        custList.add(new Customer(103, "SD", "sd@jlc.com", 333, city));
        custList.add(new Customer(104, "DS", "ds@jlc.com", 444, city));
        return custList;
    }
}
```



## 6. JLCApplicationConfig.java

```
package com.jlcindia.springboot;

import org.springframework.boot.autoconfigure.SpringBootApplication;

import org.springframework.context.annotation.Bean;
import org.springframework.web.servlet.config.annotation.*;
import com.google.common.base.Predicates;
import springfox.documentation.builders.PathSelectors;
import springfox.documentation.builders.RequestHandlerSelectors;
import springfox.documentation.service.ApiInfo;
import springfox.documentation.service.Contact;
import springfox.documentation.spi.DocumentationType;
import springfox.documentation.spring.web.plugins.Docket;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@SpringBootApplication
public class JLCApplicationConfig implements WebMvcConfigurer {

    private ApiInfo getApiDetails() {
        return new ApiInfo("JLC Bank API",
            "Customer and Account API - part of JLCBank", "1.1",
            "Free to use ",
            new Contact("Srinivas Dande", "https://www.jlcindia.com", "sri@jlcindia.com"),
            "API Under Free Licence",
            "https://www.jlcindia.com");
    }

    @Bean
    public Docket api() {
        return new Docket(DocumentationType.SWAGGER_2).select().paths(PathSelectors.any())
            .apis(Predicates.not(RequestHandlerSelectors.basePackage("org.springframework.boot")))
            .build()
            .apiInfo(getApiDetails());
    }

    @Override
    public void addResourceHandlers(ResourceHandlerRegistry registry) {
        //enabling swagger-ui
        registry.addResourceHandler("swagger-ui.html").addResourceLocations("classpath:/META-INF/resources/");
        registry.addResourceHandler("/webjars/**").addResourceLocations("classpath:/META-INF/resources/webjars/");
    }
}
```



## 8. application.properties

```
spring.application.name=my-jlc-application  
server.port=12345
```

## 9. pom.xml

```
<project ...>  
  
<modelVersion>4.0.0</modelVersion>  
<groupId>com.jlcindia.springboot</groupId>  
<artifactId>demo7-rest-swagger</artifactId>  
<version>1.0</version>  
<packaging>jar</packaging>  
<name>demo7-rest-swagger</name>  
<description>Spring - Boot - Rest-swagger</description>  
  
<parent>  
<groupId>org.springframework.boot</groupId>  
<artifactId>spring-boot-starter-parent</artifactId>  
<version>2.5.3</version>  
</parent>  
  
<dependencies>  
  
<dependency>  
<groupId>org.springframework.boot</groupId>  
<artifactId>spring-boot-starter-web</artifactId>  
</dependency>  
  
<dependency>  
<groupId>io.springfox</groupId>  
<artifactId>springfox-swagger2</artifactId>  
<version>2.4.0</version>  
</dependency>  
  
<dependency>  
<groupId>io.springfox</groupId>  
<artifactId>springfox-swagger-ui</artifactId>  
<version>2.4.0</version>  
</dependency>  
  
<dependency>  
<groupId>org.springframework.boot</groupId>  
<artifactId>spring-boot-devtools</artifactId>  
</dependency>  
  
</dependencies>
```



```
<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>

</plugins>
</build>
</project>
```

**Q1) Who is responsible for Processing REST Request at Server?**

**Ans:**

DispatcherServlet

**Q2) Who is responsible for Making REST Client Call?**

**Ans:**

RestTemplate

**Q4) Who is responsible for Marshalling/ UnMarshlling Data at Server?**

**Ans:**

DispatcherServlet

**Q3) Who is responsible for Marshalling/UnMarshlling Data at Client?**

**Ans:**

RestTemplate

**Q5) How RestTemplate will come to know about XML format/JSON Format?**

**Ans:**

Using Response Headers.

**Q6) What are the HttpMessageConverters registered with DS and RT?**

**Ans:**

```
ByteArrayHttpMessageConverter
StringHttpMessageConverter
Jaxb2RootElementHttpMessageConverter
MappingJackson2HttpMessageConverter

ResourceHttpMessageConverter
ResourceRegionHttpMessageConverter
SourceHttpMessageConverter
AllEncompassingFormHttpMessageConverter
```



**Q7) If we put @DeleteMapping and put the functionality of getting the customer in the implementation, how the annotation @DeleteMapping plays role?**

**Ans:**

You can ...but Incoming Request Http method must be delete.  
But Not Recommended to Use like this.

**Q8) Can mark getCustomerByCid() method with @DeleteMapping?**

**Ans:**

Same as Q7

**Q9) If I don't want to send entire Object ..i.e I want omit some fields in response How can I do this ?**

**Ans:**

make the fields as Transient ...  
Transient fields will not be Marshalled or UnMarshalled.

**Q10) What is the use of runtime exception and request object as parameter in the following methods?**

**Ans:**

```
ResponseEntity<MyCustomErrorResponse> handleCidNotFound(RuntimeException,  
WebRequest)  
ResponseEntity<MyCustomErrorResponse> handleCidNotFound(RuntimeException)  
ResponseEntity<MyCustomErrorResponse> handleCidNotFound( WebRequest)
```

**Q11) How Can I Customize the Message with ResponseEntityExceptionHandler?**

**Ans:**

Yes , You can.

**Q12) Can I send exception error message in header section ?**

**Ans :**

Yes , Any Data you can set to Response Headers ( in Success case or failure Case)

**Q13) Global Exception handler Vs Local Exception handler**

**Ans:**

Consider InvalidPhoneNumber exception to be thrown in multiple places like while adding phone number or updating phone number, everywhere you need to write same piece of code to avoid that global exception handler is written, you just need to throw exception from anywhere but will be handled and relevant message will be sent back to user from centralized place.

**Q14) Which exception handler is specific to controller class in REST API CASE?**

**Ans:** Local Exception handler



**Q15) What @controllerAdvice indicates to DS?**

**Ans:**

Tells about Centralized place where all the Exceptions are configured

**Q16) What happens when thrown exception is not configured?**

**Ans:**

DS will send exception stack to client

**Q17) Can I write two methods to Handle Same Exception in One Global Exception handler?**

**Ans:**

No

**Q18) Can I write Multiple Global Exception handlers?**

**Ans:**

Yes

**Q19) Can I write two methods to handle Same Exception in Two different Global Exception handlers?**

**Ans:**

Yes

**Q20) Can I create basecontroller which will have all exception handling?**

**Ans:**

No

**Q21) Can we pass multiple exception classes to @ExceptionHandler?**

**Ans:**

Yes , You can

```
@ExceptionHandler({  
    CustomerIDNotFoundException.class,InvalidPhoneNumberException.class})
```

**Q22) Can I write controller by extending base controller?**

**Ans:**

Yes , You centralize the common functionality of All the Controllers in  
BaseController

```
@RestController  
class CustomerController extends JLCBaseController{  
  
}
```

**Q23) What is the use of Swagger?**

**Ans:**

- 1) Rest API Documentation
- 2) Rest API Testing
- 3) Once the Swagger is Ready, Give Swagger Access to Client to consume Rest API.
- 4) Swagger CodeGen Tool can be used for generating Resources at Client App

**Q24) What are the IMP annotations in Swagger?**

**Ans:**

- 1) ApiModel (Resource Class)
- 2) ApiModelProperty (Resource Class Property)
- 3) ApiOperation (Controller Operation)

**Q25) Is there any provision that we can use Swagger API document to generate Java classes using Jackson libraries at client side?**

**Ans:**

Yes, You do this by using Swagger CodeGen Tool.

**Q26) Can I Generate the POJO class based on Schema available in Swagger?**

**Ans:**

Yes, You do this by using Swagger CodeGen Tool.

**Q27) How do we share schemas to Client so that Client can generate pojo classes?**

**Ans:**

Schemas are available with Swagger API, You can share them

**Q28) If we have multiple Rest API running in different Apps then all these are visible in single Swagger UI?**

**Ans:**

No, we can try *SwaggerHub*

**Q29) What is SwaggerHub?**

**Ans:**

- SwaggerHub is a platform that supports the idea of having multiple different APIs managed by a single organization or team, and we can use its back end to load these in as needed to our documentation portal.
- By using a tool like SwaggerHub, we can update and define our 'current' specifications to be shown to API consumers, while also supporting ongoing design and development.